

A Project on Transforming Third-Person RealSense Human Localisation into SLAM Map Coordinates

Abstract	2
1. Introduction	2
2. Methodology	3
2.1 Human & Teleco Detection Models	3
2.1.1 Pose-Based Human Detection	3
2.1.2 Simple Teleco & Human Class Detector for Calibration	3
2.2 2D XY Mapping via Kabsch Solver	4
2.3 Height Calibration Model	4
2.4 MQTT Communication	4
2.5 GUI for Calibration & Detection	4
3. User Manual	5
3.1 Setting Up the RealSense Camera	5
3.2 Calibrating the Rover's SLAM Pose and Aligning Coordinates	5
3.3 Calibrating the Transformation Matrices	6
3.4 Defining Boundary Coordinates	7
3.5 Defining reference point for closest human	7
3.6 Summary of calibration parameters	8
3.6 Starting the Detection Scripts	9
3.7 Finetuning Detection Parameters (Optional)	9
4. Accuracy Evaluation	10
4.1 Height Calibration accuracy	10
4.2 Coordinate Calibration accuracy	12
Table 3: Close up of Transformed data error	12
5. Conclusion	14
5.1 Reason for not using Kabsh algorithm for height calibration	14
5.2 Number of points chosen for transformation matrix	14
5.3 Summary	14

Abstract

This report presents a vision based human detection and localization system developed to support the Teleco Robot, a two part mobile social robot, at Osaka Expo 2025. Using an external Intel RealSense camera and a YOLO model, the system detects people in 3D, maps their positions into the robot's SLAM coordinate frame, and estimates their height for natural gaze control. Calibration involves a two-step process, a Kabsch-based transform for XY mapping, and a pixel to height model for vertical scaling. A built-in GUI and MQTT communication enable easy setup and real-time operation.

1. Introduction

Social robots are increasingly deployed in public spaces to greet, guide, and converse with visitors. The Teleco Robot combines a humanoid upper torso, with a facial display screen that animates expressions while it speaks, with a rover base for smooth navigation, serving as an interactive guide at Osaka Expo 2025 as shown in figure 1. To locate and engage people, the Teleco requires accurate, real-time human positions in its SLAM map and knowledge of each person's height for appropriate head-tilt and gaze.



Figure 1: Teleco mounted on the rover on the left and close up of Teleco expression on the right

This work addresses two core challenges:

1. **2D Position Mapping:** Convert RealSense measured 3D points into the robot's 2D SLAM coordinate frame.
2. **Height Estimation:** Translate pixel heights into real world human heights for dynamic gaze control.

The solution integrates two calibration algorithms, a classic Kabsch rigid-body solver and a pixel to real height curve fit, within an easy to use OpenCV GUI, and communications via MQTT.

2. Methodology

Key milestones included:

- Detecting humans and Teleco through the Yolo model
- Transforming camera into SLAM map points using a calibrated transformation matrix
- Transforming pixel-height into actual height
- Automating Calibration of transformation matrixes
- Delivering both pose and height data over MQTT

2.1 Human & Teleco Detection Models

2.1.1 Pose-Based Human Detection

We use a YOLOv11-pose model to detect people and extract keypoints. From each detection, we compute a single “ground” point for depth sampling and pixel-height measurement:

- Both-ankles midpoint (**preferred**)
- Single-ankle (**if one ankle is missing**)
- Nose-aligned bottom of the bounding box (**fallback**)

This ensures a reliable foot position even when some keypoints aren’t detected.

2.1.2 Simple Teleco & Human Class Detector for Calibration

To build the 3D–3D correspondences needed by the Kabsch solver, we only needed a bounding-box detector, not full pose keypoints.

We trained a **YOLOv11** classification model on two classes: “**teleco**” and “**person**”.

- The **Teleco** dataset was collected by recording videos of the robot at various positions and labeling its bounding boxes using CVAT-AI, an open-source annotation tool.
- The **Person** dataset was collected from public open-source datasets to ensure the model could generalize well and ignore background clutter.

The Teleco class is critical in calibration as it provides **reliable ground-truth** SLAM coordinates via MQTT, making it more accurate than relying on human poses. This allows us to pair:

- **Camera-frame 3D points** (via depth → deprojection)
- With their exact **SLAM map coordinates** (from the robot’s internal localization)

We then automatically record each camera–map pair through the calibration GUI.

2.2 2D XY Mapping via Kabsch Solver

Operators record ≥ 3 camera–map point pairs through the GUI on the video feed (camera deprojected to 3D) and pair them with SLAM poses received over MQTT. The Kabsch method then computes rotation $\mathbf{R}$ and translation $\mathbf{t}$, yielding a homogeneous transform

$$T_{map \leftarrow cam} = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

This matrix converts any new 3D camera point into the robot's global XY frame.

2.3 Height Calibration Model

We measure the robot's pixel bounding-box height at various known distances to build a dataset (d_i, p_i) . Fitting:

$$p = \frac{a}{b} + b$$

via nonlinear least squares gives parameters (a, b) . Then for any new detection:

$$h_{real} = \frac{p_{meas}}{\frac{a}{b} + b} \times H_{robotref}$$

where H_{robot_ref} is the known physical height of the reference object (the Teleco torso).

2.4 MQTT Communication

- **Calibration Script** subscribes to `/topic/.../ROVER-001/info` to receive SLAM poses.
- **Detection Script** publishes JSON messages to `human/results`, including map-frame X, Y, estimated height, and confidence—at ~5 Hz.

2.5 GUI for Calibration & Detection

An OpenCV window provides:

- A **dropdown** or **Auto-Select** button to choose the Teleco ID.
- **Record** (SPACE) and **Clear/Stop** buttons.

Real-time overlays showing counts of recorded pairs, current fit parameters, and SLAM/deprojected coordinates.

3. User Manual

This section outlines the full workflow for setting up and operating the human detection and localization system using a third-person RealSense camera and the Teleco Robot's SLAM system.

3.1 Setting Up the RealSense Camera

1. Mount the camera in a **stable**, elevated position with a **full view of the working area** as shown in figure 2.
2. **Clean the lens** to avoid depth errors from smudges or dust.
3. Connect via USB and run `realsense-viewer` on the terminal to check the color and depth streams.



Figure 2: RealSense setup on the left and Program running on the laptop on the right

3.2 Calibrating the Rover's SLAM Pose and Aligning Coordinates

1. **Connect your laptop to the same network** as the Teleco robot.
2. Open an SSH session with X11 forwarding to allow RViz to run on your local laptop:
`ssh -X rover@rover-001.local`
3. Launch the SLAM system on the rover:
`bash roverpackageserver/bash/rviz.sh`
4. Wait for the SLAM map and the robot's current position to appear in RViz.
5. **If the map does not appear:**
Check if nodes are running using: `tmux a -t rosnavigation`
 - If nodes are stuck or inactive, terminate and restart them (**Ctrl+C** inside tmux or restart the terminal session).
 - If issues persist, **reboot the entire rover**, as the LiDAR may fail to initialize properly.
6. Once SLAM is running and the map appears, verify that the robot's position is correctly aligned with its real-world location. Align with the **2D pose estimate tool** if needed. These SLAM coordinates will later be used for transformation calibration.

3.3 Calibrating the Transformation Matrices

1. Launch the **Simple YOLO model** to detect the Teleco robot in the camera feed using:
`python3 mqtt_publisher/helper_scripts/mqtt_calibration.py`
(Modify **MQTT_TOPIC** to the correct rover to receive its data)
2. In the calibration GUI, **select the Teleco id to track** like shown in figure 3 and **press the spacebar** to automatically record each pair:
 - The 3D camera coordinates (via depth deprojection)
 - The current SLAM position (received via MQTT)
3. Record **at least 3–6 point pairs** at diverse positions for reliable matrix calibration like shown in figure 4. (Make sure the **height of Teleco** is at the **maximum**)

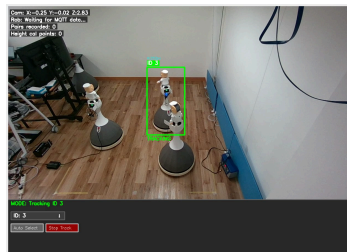


Figure 3: Select Teleco to track

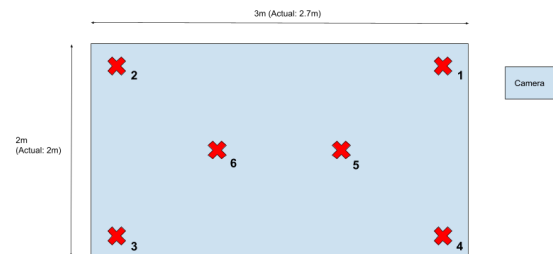


Figure 4: Optimal calibration points

4. The Kabsch algorithm computes the rigid-body transformation matrix that maps 3D camera coordinates to the 2D SLAM map by projecting all points onto the ground plane ($Z = 0$), allowing consistent XY localisation.
5. Concurrently, a curve is fitted between pixel height and real-world distance using the Teleco robot as a reference, enabling accurate estimation of human height during detection.
6. Copy paste the 2 Transformation matrices in figure 5 and 6 into `mqtt_human_detection.py` script.

```
# Position Transformation Matrix (Camera to Map coordinates)
T_MAP_CAM = np.array([
    [-0.09542898,  0.58605438, -0.80463257,  1.87720096],
    [ 0.99494907,  0.06144157, -0.05868243,  0.68282401],
    [ 0.03113944, -0.80616843, -0.59086615,  1.60601954],
    [ 0.00000000,  0.00000000,  0.00000000,  1.00000000],
], dtype=float)
```

Figure 5: Calibrated transformation matrix

```
# Height Calibration Parameters
# Model: pixel_height = a / distance + b
HEIGHT_PARAMS = [597.93191161, 78.56667181]
ROBOT_REAL_HEIGHT = 1.360 # meters
```

Figure 6: Calibrated height parameter

3.4 Defining Boundary Coordinates

1. Launch the boundary selector script using:

```
python3 mqtt_publisher/helper_scripts/boundary_coordinates.py
```

2. Click on four corners in the camera feed to define the polygonal detection region.
3. Press **ESC** to save and apply the boundary.
4. Only detections inside this region will be processed to reduce noise and false positives.
5. Copy paste the boundary coordinates as seen in figure 7 into mqtt_final.py script.

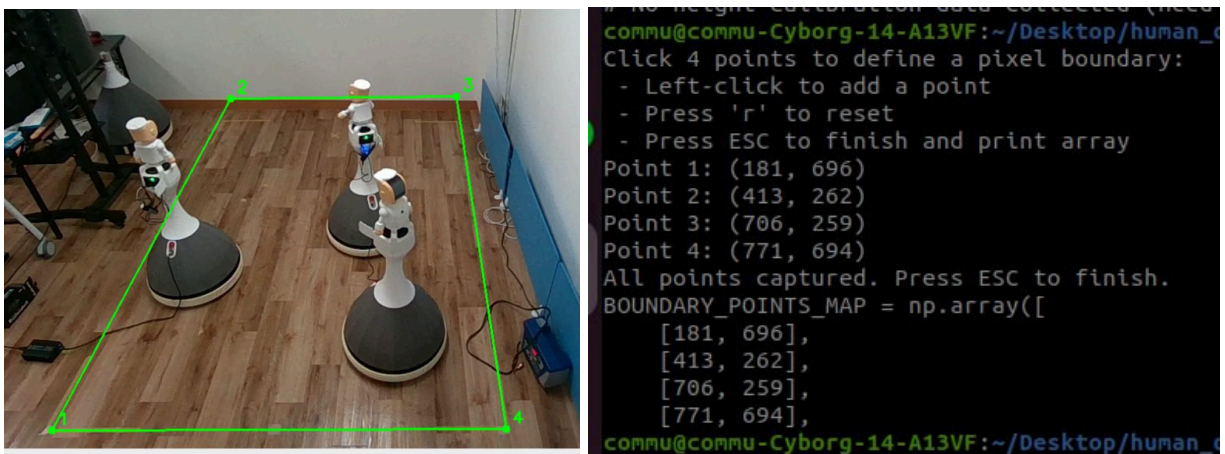


Figure 7: Chosen boundary points on the left and Generated Coordinates on the right

3.5 Defining reference point for closest human

Operators can define a **custom reference point** on the slam map within the camera view. The system then continuously compares the positions of all detected humans and identifies the one **closest to this reference point**. Only this person's transformed coordinates and height are sent to the robot, allowing it to focus on interacting with the nearest individual to the specified location.

```
DEFAULT_SLAM_POINT = (-0.9, 0.0)
```

3.6 Summary of calibration parameters

Figure 8 below shows a complete summary of all the key parameters that must be calibrated and manually pasted into the final detection script (mqtt_human_detection.py). These include:

- The **boundary coordinates** for filtering detections
- The **transformation matrix** for mapping 3D camera points to SLAM coordinates
 - Modify **MQTT_TOPIC** to the correct rover to receive its data
 - Make sure the **height of Teleco** is at the **maximum** before calibration
- The **height calibration** parameters used for estimating human height
- The **SLAM reference point** for selecting the closest human

```
# 4-point pixel boundary coordinates (boundary_coordinates.py)
# TODO: Replace these with your actual calibration results
BOUNDARY_POINTS_MAP = np.array([
    [175, 699],
    [407, 261],
    [705, 258],
    [775, 694],
], dtype=np.int32)

# Transformation matrix calibration parameter (mqtt_calibration.py)
# TODO: Replace these with your actual calibration results
T_MAP_CAM = np.array([
    [-0.10083076, 0.57155089, -0.81434804, 1.97670059],
    [0.99490299, 0.05702716, -0.08316211, 0.65838077],
    [-0.00109143, -0.81858260, -0.57438779, 1.48790371],
    [0.00000000, 0.00000000, 0.00000000, 1.00000000],
], dtype=float)

# Height calibration parameters (mqtt_calibration.py)
# TODO: Replace these with your actual calibration results
HEIGHT_PARAMS = [552.58464239, 109.35373113]
ROBOT_REAL_HEIGHT = 1.360 # meters

# TODO: Replace these with actual slam reference point coordinates
DEFAULT_SLAM_POINT = (-0.9, 0.0)
```

Figure 8: All the parameters that needs to be calibrated in the detection script

3.6 Starting the Detection Scripts

Run the main detection script:

1. `python3 mqtt_publisher/mqtt_human_detection.py`
2. The script will:
 - Detect humans using YOLOv11-pose
 - Extract keypoints and calculate ground position
 - Transform camera 3D points into SLAM XY coordinates
 - Estimate human height using the calibrated scaling model
 - Filter detections to show only inside the custom boundary
 - Publish results to MQTT under: human/results
3. The GUI as shown in figure 9 will display:
 - Detected humans and Teleco with bounding boxes (Closest human to reference point in yellow, rest in pink)
 - Transformed SLAM coordinates
 - Real-time height estimates

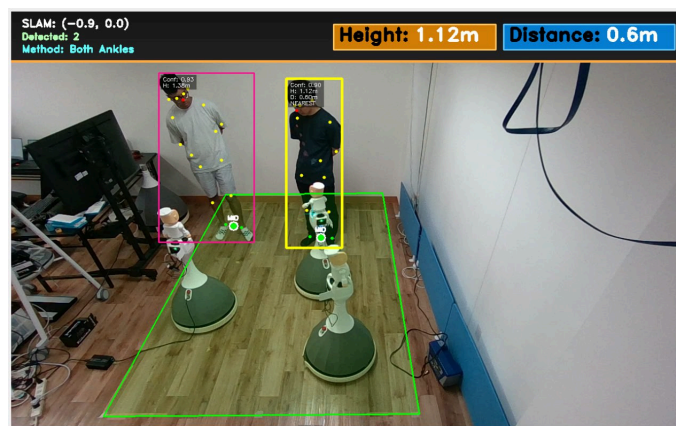


Figure 9: Detection scripts showing estimated height and distance to reference point

3.7 Finetuning Detection Parameters (Optional)

1. If detection is unreliable (e.g., false positives, jittery boxes), tweak the following script parameters:
 - **CONF_THRESH = 0.4** # Detection confidence threshold
 - **IOU_THRESH = 0.5** # Non-max suppression threshold
2. To use a different model, update the **POSE_MODEL_PATH** and **YOLO_MODEL**.
3. If lighting or scene setup changes significantly, repeat both the transformation and height calibration for best accuracy.

4. Accuracy Evaluation

4.1 Height Calibration accuracy

The height calibration accuracy was evaluated using nine marked positions within the camera's field of view. At each marker, the measured height was recorded five times, and the average, standard deviation (SD), and error compared to the actual reference height of 1.53 m were computed (see Table 1). The results show that while the calibration model effectively reduces large errors, minor discrepancies remain due to several factors.

Marker	Transformed Height (m)					Average (m)	SD	Actual (m)	Error (m)
	1	2	3	4	5				
1	1.5	1.45	1.43	1.48	1.5	1.47	0.03	1.53	0.06
2	1.52	1.54	1.53	1.52	1.54	1.53	0.01	1.53	0
3	1.5	1.5	1.49	1.48	1.48	1.49	0.01	1.53	0.04
4	1.53	1.53	1.51	1.53	1.52	1.52	0.01	1.53	0.01
5	1.55	1.53	1.5	1.52	1.54	1.53	0.02	1.53	0
6	1.51	1.5	1.5	1.48	1.5	1.5	0.01	1.53	0.03
7	1.44	1.45	1.45	1.46	1.45	1.45	0.01	1.53	0.08
8	1.47	1.45	1.45	1.45	1.45	1.45	0.01	1.53	0.08
9	1.42	1.44	1.43	1.43	1.42	1.43	0.01	1.53	0.1

Table 1: Height data collected at nine marked positions

One major source of error is inconsistent posture of the test subject. Small variations such as looking slightly up or down alter the detected keypoints (e.g., nose or ankles), which directly affects the measured pixel height and consequently the real-world height estimation. Another observed issue arises from camera perspective distortion, particularly near the frame edges, where the tilted camera introduces angular distortion. For example in figure 10, Marker 1 (bottom-right corner) shows a slightly higher error (0.06 m), as seen in the image, while Marker 5 (center) yields zero error due to optimal **perspective alignment**.

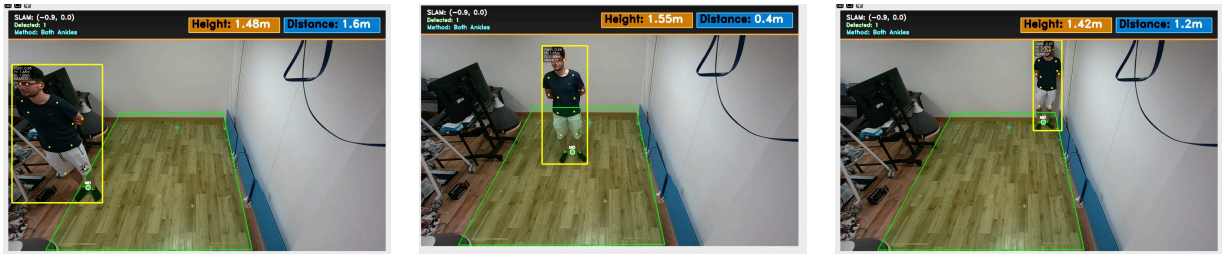


Figure 10: Marker 1 (Left), Marker 5 (Middle), Marker 9 (Right)

Additionally, objects farther from the camera appear smaller, which makes the height estimation more sensitive to keypoint inaccuracies. This effect is noticeable at markers 8 and 9, where the error increases to 0.08 m and 0.10 m respectively. Despite this, the calibration remains sufficiently accurate for the intended gaze-control application, with errors remaining under 10 cm in most scenarios. Manual offset correction could be applied if needed to fine-tune the system.

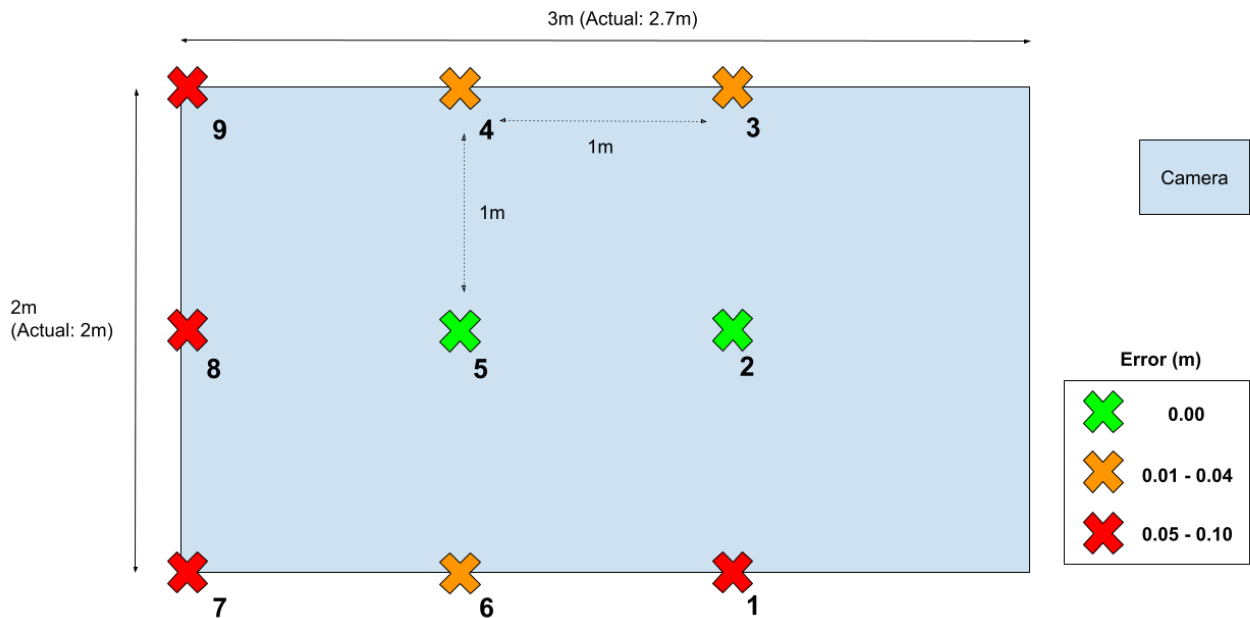


Figure 11: Marker placements on the map

The findings suggest that camera placement and alignment are critical to reducing perspective-based errors. Keeping the camera upright and centered over the area of interest significantly improves the reliability of the calibration. Future improvements may involve collecting additional calibration samples across varying distances and angles, or applying a secondary correction model (e.g., perspective correction or polynomial fitting) to further minimize edge-case errors.

4.2 Coordinate Calibration accuracy

To evaluate the accuracy of the transformation matrix, we tested the same nine marked positions across the scene as in figure 11 and recorded the transformed (camera → SLAM) coordinates five times per marker. For each position, the transformed X and Y coordinates were compared against the ground-truth SLAM coordinates received via MQTT (see Table 2). We then computed the average, standard deviation (SD), and absolute error for each axis.

Marker	Class	SLAM Coordinates (m)		Class	Transformed Coordinates (m)						Average (m)	SD (m)	Error (m)
		Axis	SLAM		Axis	1	2	3	4	5			
1	Teleco	X	0.33	Human	X	0.31	0.3	0.32	0.3	0.32	0.31	0.01	0.02
		Y	-0.99		Y	-0.96	-1	-0.95	-0.97	-0.97	-0.97	0.02	0.02
2	Teleco	X	0.18	Human	X	0.13	0.15	0.17	0.16	0.13	0.15	0.02	0.03
		Y	-0.02		Y	-0.06	-0.02	-0.02	-0.04	-0.04	-0.04	0.02	0.02
3	Teleco	X	0	Human	X	0.05	0.02	0.08	0.05	0.05	0.05	0.02	0.05
		Y	0.93		Y	0.91	0.92	0.92	0.96	0.93	0.93	0.02	0
4	Teleco	X	-1.01	Human	X	-1	-1.04	-1.06	-1.08	-1.03	-1.04	0.03	0.03
		Y	0.8		Y	0.82	0.81	0.81	0.81	0.8	0.81	0.01	0.01
5	Teleco	X	-0.9	Human	X	-0.95	-0.94	-0.95	-0.94	-0.98	-0.95	0.02	0.05
		Y	-0.24		Y	-0.17	-0.19	-0.18	-0.2	-0.16	-0.18	0.02	0.06
6	Teleco	X	-0.75	Human	X	-0.77	-0.72	-0.75	-0.72	-0.7	-0.73	0.03	0.02
		Y	-1.18		Y	-1.18	-1.15	-1.19	-1.13	-1.11	-1.15	0.03	0.03
7	Teleco	X	-1.75	Human	X	-1.74	-1.82	-1.75	-1.83	-1.79	-1.79	0.04	0.04
		Y	-1.31		Y	-1.32	-1.34	-1.28	-1.34	-1.29	-1.31	0.03	0
8	Teleco	X	-1.9	Human	X	-1.92	-1.9	-1.9	-1.94	-1.96	-1.92	0.03	0.02
		Y	-0.3		Y	-0.3	-0.3	-0.32	-0.36	-0.32	-0.32	0.02	0.02
9	Teleco	X	-2	Human	X	-2	-2.01	-2.06	-2.05	-1.96	-2.02	0.04	0.02
		Y	0.62		Y	0.66	0.7	0.68	0.65	0.70	0.67	0.02	0.05

Table 2: Transformed data collected at nine marked positions

Marker	Axis	Class	SLAM	Class	Average (m)	SD (m)	Error (m)
1	X	Teleco	0.33	Human	0.31	0.01	0.02
	Y	Teleco	-0.99		-0.97	0.02	0.02
2	X	Teleco	0.18	Human	0.15	0.02	0.03
	Y	Teleco	-0.02		-0.04	0.02	0.02
3	X	Teleco	0	Human	0.05	0.02	0.05
	Y	Teleco	0.93		0.93	0.02	0
4	X	Teleco	-1.01	Human	-1.04	0.03	0.03
	Y	Teleco	0.8		0.81	0.01	0.01
5	X	Teleco	-0.9	Human	-0.95	0.02	0.05
	Y	Teleco	-0.24		-0.18	0.02	0.06
6	X	Teleco	-0.75	Human	-0.73	0.03	0.02
	Y	Teleco	-1.18		-1.15	0.03	0.03
7	X	Teleco	-1.75	Human	-1.79	0.04	0.04
	Y	Teleco	-1.31		-1.31	0.03	0
8	X	Teleco	-1.9	Human	-1.92	0.03	0.02
	Y	Teleco	-0.3		-0.32	0.02	0.02
9	X	Teleco	-2	Human	-2.02	0.04	0.02
	Y	Teleco	0.62		0.67	0.02	0.05

Table 3: Close up of Transformed data error

As shown in Table 3 on the left, the coordinate transformation maintains a high level of accuracy. Most transformation errors across the X and Y axes are within 0.02 m to 0.06 m, which confirms the Kabsch-calculated transformation matrix is consistent and precise across the tested region.

Minor deviations can arise from clicking inaccuracies during calibration, slight detection shifts due to human pose variation, or sensor noise in depth estimation. Nevertheless, the overall low error values indicate that the coordinate mapping between the camera and the SLAM frame is reliable and sufficiently accurate for robot-human interaction tasks at the Osaka Expo 2025.

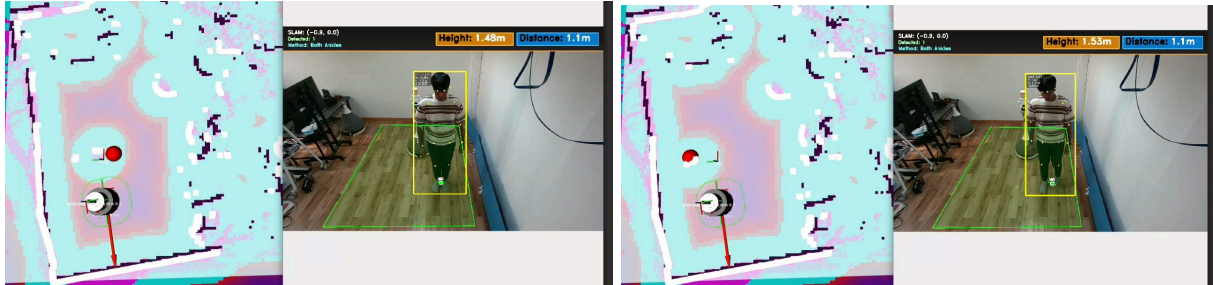


Figure 12: Accurate Marker alignment with human legs on lidar

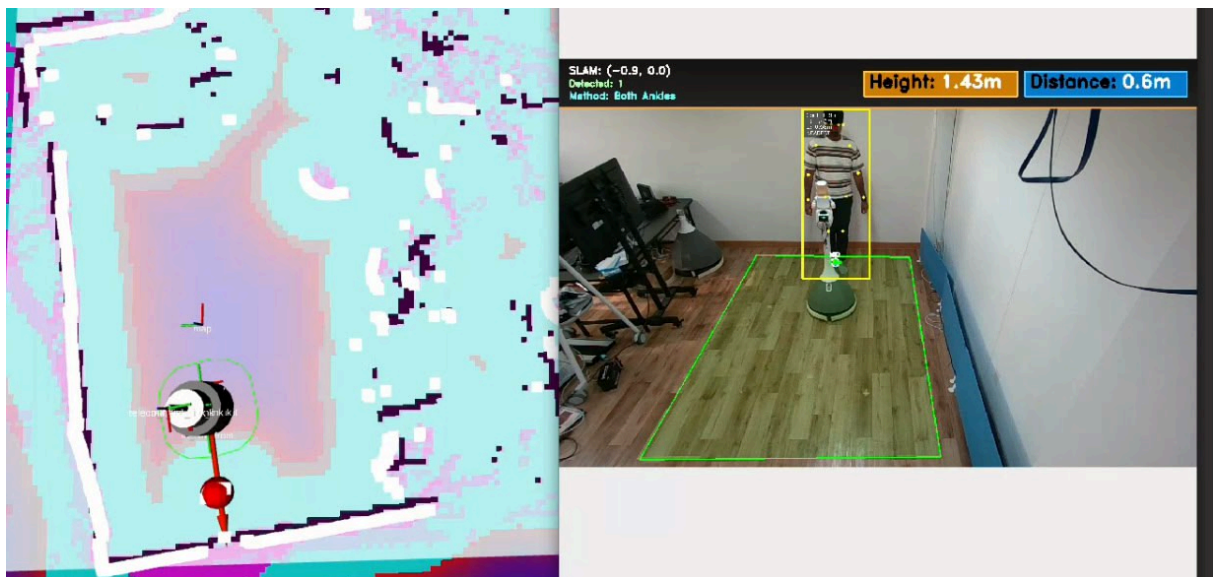


Figure 13: Accurate Human detection behind Teleco

The visual results in Figures 12 and 13 further reinforce this accuracy. The markers generated in RViz, based on transformed human detection coordinates, align precisely with the person's legs as captured by the Teleco robot's LiDAR scan. This alignment showcases the precision of the 3D camera to 2D SLAM transformation and confirms the effectiveness of the system's depth-based localization and pixel-to-height calibration. Even when the human is partially occluded behind the robot, the detections remain consistent and spatially accurate, highlighting the robustness of the pipeline for real-time deployment.

5. Conclusion

5.1 Reason for not using Kabsch algorithm for height calibration

While the Kabsch algorithm is well-suited for 3D-to-3D rigid-body transformations, such as aligning camera coordinates with SLAM map coordinates, it is not practical for height calibration, which involves estimating a variable scalar rather than aligning fixed point sets. Human height changes from person to person and varies with distance, making rigid transformation unsuitable. Instead, we use the Teleco robot's known constant height as a reference object to build a pixel to height scaling model. By recording its pixel height at different distances, we fit a curve that allows accurate and consistent height estimation across the scene. This approach simplifies calibration, avoids noisy Z-values, and provides a more operator-friendly and robust solution for the 2025 Osaka Expo.

5.2 Number of points chosen for transformation matrix

During testing, multiple transformation matrices were generated using different numbers of calibration point pairs (ranging from 3 to 9). Surprisingly, there was **no clear correlation between the number of points and the accuracy of the resulting transformation**. In several cases, matrices calibrated with just **3 high-quality, well-distributed points** actually performed better than those using 9 points, which could introduce inconsistencies if some points were noisy or too clustered.

This led to the conclusion that the **quality and spatial distribution** of the selected calibration points matters far more than quantity. As long as the chosen points are spread across the entire observable map area and are free from depth noise or detection inaccuracies, even a minimal set is sufficient to compute an accurate and robust transformation matrix using the Kabsch algorithm. This simplifies the calibration process and makes it more efficient for field deployment.

5.3 Summary

Overall, the system demonstrates accurate and consistent performance in both coordinate mapping and height estimation. The dual calibration strategy, rigid-body Kabsch transformation for XY mapping and pixel-based scaling for height, proves to be effective and practical. By leveraging the known dimensions and SLAM position of the Teleco robot, we enable a reliable real-time human localization system suitable for gaze interaction at the Osaka Expo 2025.